TekPractice — The Educative Game (Vision & Structure)

Council-validated (Tier 3, 82%). The "never done before" training game inside Teknest, for the BTS Métiers du son.

0. The one-line + the hook

A calm, white, gallery-beautiful sound-engineering game where you learn the theory, practise on guided levels, sit real exam-style tests, and then build *any* audio system — grabbing real gear from category shelves and dropping it into a studio, a théâtre, an open-air stage, a radio booth, a restaurant, a reportage van — while a live rules engine validates and teaches every connection, and a progression brain shows your BTS exam-readiness climb.

Why it's never been done: nobody combines **real gear + real signal-flow rules + an unlimited-feeling environment sandbox + référentiel-mapped analytics + a peer-mentor voice**, in a **premium white aesthetic**, aimed squarely at passing a real diploma (theory *and* practical). Competitors sell PDFs or dark hobby mixing courses. This is a *reactive smart sound simulation that is also a curriculum*.

1. The core idea

One engine (your existing catalog: 26 connectors, 16 signals, 24 real devices, rules R1–R8) powers four modes that share one progression brain. "Without limits" = **combinatorial freedom inside a real, correct system** — not infinite content. That reframing is what makes it both unprecedented and fast.

2. The four modes (one spine, one analytics brain)

A. Learn (theory · maps to E3/PTES). Micro-lessons + retrieval quizzes from the Tekskol courses (acoustique, électricité, transducteurs, audionumérique, traitement du signal). Short, spaced, worked-examples-first.

B. Levels (guided practice · maps to E4). Scaffolded scenarios in a *specific* environment with a brief ("sonorise ce plateau TV"). The rules engine teaches via the R1–R8 toasts. Difficulty rises; hints fade. This is where fundamentals are drilled safely.

C. Tests (exam mode · maps to E3/E4/E51). Timed, exam-format, **no hints**, scored against the référentiel. Pure retrieval + assessment = the readiness check. Mirrors the real épreuves.

D. Sandbox / Freeplay (the soul · "without limits"). Pick an environment → open the **category shelves** (Micros, Consoles, Monitors, Stageboxes/IO, Codecs, Patchbays, Câbles…) → **grab & drop** any gear → wire a full reactive system → live validation + coaching → **save & share your rig**. Day-one = *guided sandbox* (limited palette + a brief); full unlimited palette unlocks as mastery grows (the council's resolution of freedom vs overwhelm).

+ Coach layer (your touch + psychology). Woven through all modes: your first-person "voici comment j'ai réussi / ce sur quoi j'ai focalisé", calm anti-burnout framing, "mistakes are data" (tied to the rules), streaks and small wins, spaced-practice nudges. Later: the AI coach (ai-integrate, RAG-grounded on your docs).

3. Environments ("plateau / extérieur / restaurant / théâtre / open-air / radio / studio / …")

Each environment = a **lightweight scene preset**: a backdrop + baked lighting + a starting brief + a relevant gear subset. This is the trick that makes "without limits" cheap and fast — new environments are **data + a background**, never new engine code. Launch set: Studio son · Studio radio (self-op) · Plateau TV · Théâtre · Plein-air/concert · Reportage extérieur (ENG) · Événementiel/restaurant. Add more forever, at near-zero cost.

4. A-to-Z page/screen flow (inside Tekskol)

1. **Onboarding** — your BTS date + option Son, a 2-min skill self-check → a personalised path.
2. **Home / Dashboard** — exam-readiness radar, the single "next best activity", streak, your tip-of-the-day.
3. **Learn** — lesson → quiz → mark mastered (spaced review scheduled).
4. **Levels** — environment brief → build → rules teach → win screen with mistake summary.
5. **Tests** — exam timer → build/answer, no hints → scored readiness report.
6. **Sandbox** — environment picker → category shelves → grab/drop/wire → live coach → save/share.
7. **Coach / Mentor** — your advice library + focus guides + the psychology support.
8. **Progress / Stats** — deep analytics (below).
9. **Profile / Certificates** — ties into the Tekskol certificate system.

5. Progression & statistics (the brain — never vanity)

- **Exam-readiness, not XP.** A radar per **bloc de compétences (BC01–BC04)** and per **épreuve (E3/E4/E51)**. You literally watch your readiness rise.
- **Mistake-history → weak-spot targeting.** Every R1–R8 error is logged; the system recommends the next activity that attacks your weakest spot (your own success metric: errors trending to zero).

- **Spaced repetition** for theory; **mastery gates** unlock sandbox freedom and harder levels.
- **Signals tracked:** time-to-mastery, streak, per-domain accuracy, error-type frequency, sandbox complexity reached, test scores vs readiness.
- **Psychologically safe:** progress framed as growth; no punishing leaderboards; calm, encouraging tone.

6. The white artistic direction

Gallery-white, calm, premium — think Teenage Engineering × Apple × a real studio. Devices are beautiful objects on a soft white floor with gentle contact shadows; cables are elegant catenaries; UI is minimal, typographic, generous whitespace, one accent colour, subtle motion. White = lowers anxiety, signals professionalism, and makes the real-gear models the hero. This is a deliberate anti-thesis to cluttered dark "gamer" sims — and a big part of the "never seen before".

7. Pros / cons

Pros	Cons / risks
Unifies theory + practical for a real diploma — no competitor does this	Scope is large; "without limits" can balloon if not bounded by the catalog
Reuses the engine you already built (fast to reach)	Content authoring (environments, levels, briefs) is ongoing work
Sandbox + share = motivation, moat, and marketing (UGC)	Perf risk if many devices/cables on screen — needs budgets/LOD
White art direction = unique, calm, premium, low-anxiety	3D on low-end student laptops — needs a graceful 2D/low fidelity fallback
Référentiel-mapped stats = provable exam value → subscriptions	Assessment rigor must be real, or "test mode" loses trust
Your student voice = authentic self-efficacy, hard to copy	Your time to author the mentor content

8. Vision stress-test (from every direction)

- **Pedagogy:** ✅ if play feeds the same rules + analytics (sandbox = assessment), and freeplay is mastery-gated. ❌ if it becomes fun-without-transfer.
- **Engineering:** ✅ if bounded by catalog + preset environments, content-as-data, perf budgets. ❌ if "literally infinite" forces engine rewrites.
- **Psychology:** ✅ white calm + scaffolding + productive-failure + small wins + your mentorship. ❌ if it uses anxiety-driven gamification (leaderboards, punishing timers everywhere).

- **Business:** ✅ free taster (1 environment, a few levels) → subscription for full sandbox + tests + analytics; certificates tie-in; shared rigs drive growth. ⚠ if the free tier gives away the whole value.
- **UX:** ✅ one "next best action" so users never feel lost. ⚠ if the sandbox palette overwhelms day one (hence guided-sandbox first).
- **Domain fidelity:** ✅ real gear + real rules + your BTS expertise reviewing content. ⚠ if any signal-flow rule is subtly wrong (you are the gate).

9. How to develop it (phased — ship value early)

- **P0 (now):** you already have the engine + A1 + B1/C1/D1 logic. Wrap it in the **white art direction** (Designer) and the **dashboard + readiness stats** (Developer).
- **P1 — Modes spine:** Learn (pull from Tekskol courses) + Levels (existing) + Tests (exam mode) sharing the progression brain.
- **P2 — Sandbox v1:** category shelves + grab/drop + free graph + live coach, in 2–3 environments (studio, radio, plateau). Guided-sandbox gating.
- **P3 — Environments + sharing:** add théâtre / open-air / ENG / événementiel as presets; save & share rigs.
- **P4 — AI coach + mentor library:** ai-integrate, RAG-grounded on your ingested BTS docs + your advice.
- **P5 — Polish, perf, low-end fallback, community.**

10. Technical structure for smooth / clean / fast

- **Reuse the engine.** Babylon + `catalog.json` + ConnectionGraph + RuleEvaluator + LevelRunner already exist. Sandbox = a level with **no requiredChain** (free graph, same rules). Zero engine rewrite.
- **Everything is data.** Environments, levels, briefs, category shelves = JSON. New content ≠ new code (the 3-layer rule).
- **One progression store.** Extend the existing `ProgressStore` (swappable interface) → référentiel-mapped readiness + mistake history. localStorage now, backend later.
- **Performance budget:** instanced + lazy-loaded glTF (Draco/KTX2 already done), environment = light backdrop + baked lighting, LOD, cap active devices/cables, code-split Babylon (done), degrade catenary→line under load, and a **2D/low-fidelity fallback** for weak laptops.
- **One design system** (Designer agent) shared by site + game so `teknest.fr` and the game feel like one product.
- **Clean separation:** Content (data) / Engine (Babylon+rules) / Assets (glTF) / Progress (store) / Coach (AI later) — each testable alone.

11. Your student-voice layer (the irreplaceable touch)

A recurring "Conseils d'Oscar" presence: what to focus on per épreuve, the traps you fell into, your revision rhythm, how the practical exam really feels, morale in the tough weeks. Authentic, first-person, calm. This is the trust and self-efficacy layer no competitor and no

AI can fake — and it should appear contextually (a tip on the exact topic you're studying), not as a wall of text.

12. The honest bottom line

The vision is strong and, crucially, **buildable on what you already have** — because the "never done before" part isn't new tech, it's the *combination*: real system + unlimited-feeling sandbox + référentiel analytics + white calm design + a real graduate's voice. Keep "without limits" bounded by the catalog and cheap environment presets, make the sandbox feed the same analytics as the tests, and gate freedom behind mastery. Do that and it's unprecedented, effective, and fast.